

DS	Distributed Systems
26/27	Water Management
	Tutorial: Introduction to Railway in a CI/CD setup

Preamble

Goal: get familiar with Railway before using it in the Water_Management practical.

Prerequisites

Before starting, students must know about and have the following:

- **GitHub account**, active and accessible from the browser.
- **Account on railway.app**, preferably created with GitHub login (makes connecting repos easier).
- **VS Code installed**, with the Source Control panel familiar to you (or Git from the command line, if preferred) — used to initialize the repo, make commits, and publish it to GitHub.
- **Python 3 installed** on the machine, needed both for the block 2 example and for the Kafka test script in block 3.
- **Basic terminal/command-line knowledge**: navigating between folders and running commands like `pip install`.
- **Minimal Docker knowledge**: understanding what a Dockerfile is and what it's for (you don't need Docker installed locally, since Railway builds the image in the cloud, but it's recommended).
- **Concept of environment variables** (such as `PORT`), since Railway uses them to configure services at runtime.

1. What is Railway

Railway is a cloud deployment platform (PaaS) designed to get applications and services up and running without manually managing servers.

Key ideas:

- **Project** = logical container where you group several services (for example: your app + a database + Kafka). Services within the same project can see each other over an internal private network.
- **Service** = a deployed piece: it can come from your own code (a GitHub repo, with or without a Dockerfile) or from a template already packaged by Railway or the community (databases, Kafka, Redis, etc.).
- **Automatic build**: if you connect a GitHub repo, every git push rebuilds and redeploys the service on its own — it's the "CD / Continuous Deployment" already built in, with nothing else to configure.
- **Network**: each service has an internal URL/host (`name.railway.internal`) to talk to other services in the project without going out to the internet, and optionally a public domain or TCP Proxy to be accessible from outside (for example, from your lab PCs).

In short: Railway replaces "having your own server + Docker + Nginx + manual CI (Continuous Integration)" with an interface where you connect a repo or choose a template and your service is already running and accessible.

2. Deploying a "Hello World" with CI/CD (VS Code → GitHub → Railway)

2.1 Preparing the local project

Any language works; minimal example in Python:

```
# server.py
import os
from http.server import BaseHTTPRequestHandler, HTTPServer

port = int(os.environ.get("PORT", 3000))

class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        self.send_response(200)
        self.send_header("Content-Type", "text/plain")
        self.end_headers()
        self.wfile.write(b"Hello World from Railway!")

print(f"Listening on port {port}")
HTTPServer(("0.0.0.0", port), Handler).serve_forever()

# Dockerfile
FROM python:3.12-alpine
WORKDIR /app
COPY . .
EXPOSE 3000
CMD ["python", "server.py"]
```

Important: In this example, the server must listen on the port Railway specifies via the `PORT` environment variable, not on a fixed port — Railway assigns it dynamically.

2.2 Pushing the code to GitHub from VS Code

1. Open the project folder in VS Code.
 2. Source Control panel (branch icon, Ctrl+Shift+G) → Initialize Repository.
 3. Write a commit message (e.g. "Initial Hello World") → Commit.
 4. Publish Branch button → choose Publish to GitHub → public or private repo, name e.g. railway-hello-world.
- VS Code creates the repo on GitHub and uploads the code in a single step.

2.3 Connecting Railway to the repo

1. In railway.app → New Project → Deploy from GitHub repo.
2. Authorize GitHub access (only the first time) and select railway-hello-world.
3. Railway detects the Dockerfile and builds the image automatically.
4. In Settings → Networking → Generate Domain you get a public URL like railway-hello-world-production.up.railway.app.
5. Open it in the browser: it should show "Hello World from Railway!".

2.4 Testing the CI/CD cycle

1. Change the message text in server.py (for example, add your name).
2. In VS Code: Source Control → commit message → Commit → Sync Changes (this pushes it).
3. In Railway, the Deployments tab: you'll see a new build start on its own, without touching anything manually.
4. When it finishes, reload the public URL: it should show the updated text.

With this, the VS Code → GitHub → Railway cycle is demonstrated end to end.

3. Deploying Kafka on Railway and testing remote access

3.1 Deploying Kafka from a template

1. Within the same project (or a new one) → + New → Template → search for Kafka.
2. Choose the official apache/kafka template (KRaft mode, no ZooKeeper — simpler to manage).
3. Railway deploys the broker and automatically creates a persistent volume for the data.

3.2 Exposing it externally with TCP Proxy

Kafka uses the native TCP protocol, not HTTP, so it isn't exposed with a regular public domain:

1. Kafka service → Settings → Networking → TCP Proxy → + Add.
2. Specify the broker's internal port (usually 9092).
3. Railway assigns a public host and port, like xxxxx.proxy.rlwy.net:19097 — this is the endpoint external clients will use (lab machines, your PC, etc.).

3.3 Testing remote access from an external machine

From any PC with Python installed (it doesn't need to be the same one running Railway):

```
pip install kafka-python

# test_kafka.py
from kafka import KafkaProducer, KafkaConsumer

BOOTSTRAP = "xxxxx.proxy.rlwy.net:19097" # replace with the actual host:port of the TCP Proxy

# Producer: send a test message
producer = KafkaProducer(bootstrap_servers=BOOTSTRAP)
producer.send("test-topic", b"Hello from a remote machine")
producer.flush()
print("Message sent.")

# Consumer: read it back
consumer = KafkaConsumer(
    "test-topic",
    bootstrap_servers=BOOTSTRAP,
    auto_offset_reset="earliest",
    consumer_timeout_ms=5000
)
for msg in consumer:
    print("Received:", msg.value.decode())
```

If the sent message is received back, the broker is correctly accessible from outside Railway — exactly the same mechanism that will later be used by WM_WS and WM_FO from the lab PCs.

3.4 Alternative check from the command line

With Kafka installed locally (even just the binaries, without running your own broker), it also works to run the scripts and utilities that let you access its features:

```
kafka-topics.sh --bootstrap-server xxxxx.proxy.rlwy.net:19097 --list
```

It should return the list of existing topics (empty if it's the first time) without giving a connection error.